

Adaptive Gradient Algorithm with Momentum based on Spatiotemporal Second-order Dynamical Model

1st Shahrzad Mahboubi and 2nd Hiroshi Ninomiya

Dept. Informatics, Shonan Institute of Technology, Fujisawa, Japan

{1st shaa, 2nd ninomiya}@info.shonan-it.ac.jp

Abstract—Optimization methods often face a trade-off between the fast convergence of second-order methods and the low computational cost of first-order methods. Motivated by the need to bridge this gap, this paper proposes a novel optimization algorithm called the Adaptive Dynamical Momentum Newton Algorithm (ADINA), based on a spatiotemporal second-order dynamical model incorporating both momentum and an adaptive learning rate. The proposed ADINA integrates the momentum acceleration and adaptive learning rate of Adam with the quadratic convergence properties of Newton method while eliminating the need for explicit Hessian computations. By dynamically adjusting step sizes according to the gradient norm, the proposed method achieves rapid convergence and high-precision solutions while maintaining computational efficiency similar to first-order methods. Extensive experiments on function optimization, regression, and classification tasks demonstrate that ADINA consistently outperforms existing first- and second-order optimization algorithms in terms of convergence speed and solution quality. Moreover, its practical effectiveness is validated through deep learning experiments conducted with TensorFlow, confirming its superiority over conventional built-in optimizers.

Index Terms—optimization algorithm, spatiotemporal second-order dynamical model, ordinary differential equation, Adam, neural networks, deep learning

I. INTRODUCTION

Consider a minimization problem:

$$\min_{x \in \mathbb{R}^d} f(x), \quad (1)$$

where $x \in \mathbb{R}^d$ denotes the parameter vector to be optimized. In recent years, optimization problems have become increasingly prevalent across various domains, leading to the development of numerous algorithms for finding optimal solutions. One of the most prominent applications is deep learning in the field of artificial intelligence. The objective function of deep learning is given by

$$f(x) = \frac{1}{P} \sum_{p=1}^P f_p(x), \quad (2)$$

where P denotes the number of training samples, and $f_p(x)$ is the error of the p^{th} sample. For the above problem, optimization methods typically employ gradient-based approaches described by iterative formulas (difference equations). Gradient-based methods are classified into first- and second-order methods depending on the order of derivatives used. First-order methods update parameters in the gradient direction of the objective function, with notable examples including gradient descent (GD), the momentum method, which incorporates an

inertia term, and Adam [2], which combines adaptive learning rates with the momentum method, and is one of the most commonly used optimizers in deep learning [1]. However, first-order methods often struggle to achieve accuracy and speed when optimizing highly nonlinear functions [3] because they stagnate at plateau or saddle points. On the other hand, second-order methods address this limitation by utilizing the second derivative of the objective function, as seen in the Newton and quasi-Newton methods (QN) [4]. These second-order approaches typically escape from plateaus and saddle points more efficiently than first-order methods due to their ability to leverage curvature information.

Parallel developments have emerged from studying optimization algorithms through the lens of ordinary differential equations (ODEs) [5]–[12]. This perspective enables a theoretical guarantee of convergence by analyzing the solution trajectories of the corresponding ODEs. These approaches reveal that most iterative optimization algorithms effectively implement the explicit Euler numerical integration method for the numerical integration of ODEs. A significant contribution to this framework was the Dynamical Inertial Newton (DIN) method [8], which was proposed as a spatiotemporal second-order dynamical model combining Newton and the momentum methods. This model incorporates temporal dynamics through momentum-derived second-order time derivatives and spatial dynamics through the second-order partial derivatives with respect to the parameter vector, commonly referred to as the Hessian in the Newton method. While this combination enables both acceleration and improved accuracy, the inclusion of the Hessian demands substantial computational resources. DIN was reformulated to address this computational burden by introducing an auxiliary variable, transforming the second-order ODEs involving the Hessian into a system of first-order ODEs [8]. Extending this idea further, INNA was proposed for training Neural Networks (NNs) [11], eliminating the computational burden of the Hessian while preserving the second-order dynamical structure. However, despite incorporating momentum, INNA's acceleration capabilities remain limited. Furthermore, INNA cannot directly incorporate bias correction and adaptive learning rate, which are widely used to accelerate various algorithms. It was necessary to explore alternative formulations that integrate these mechanisms. One promising direction arises from recent studies that interpret adaptive optimization algorithms, such as Adam, through the lens of ordinary differential equations (ODEs) [12]. In partic-

ular, the dynamical system representation of Adam provides a framework for embedding momentum, bias correction, and adaptive learning rates into the second-order ODEs.

Based on this observation, this paper proposes a new reformulation of the DIN framework that incorporates the adaptive dynamics of Adam while preserving the second-order properties. By implementing a new transformation to DIN that can apply Adam's dynamics, we propose a novel optimization method that combines Adam's acceleration performance and stability while holding the characteristics of second-order methods. Based on this observation, this paper proposes Adaptive momentum estimation in DIN Algorithm (ADINA), which reformulates the spatiotemporal second-order dynamical model as a system of first-order ODEs incorporating Adam's mechanisms. By combining the advantages of Newton-based methods and Adam's adaptive mechanisms, ADINA can be expected to escape plateau regions or saddle points more rapidly and achieve solutions with higher accuracy in less time. ADINA can also achieve computational efficiency by eliminating Hessian calculations while ensuring robust performance through Adam-inspired bias correction and adaptive learning rate adjustments. The effectiveness of ADINA is demonstrated through comprehensive experiments across three optimization tasks: optimization of functions, regression problems, and classification problems, comparing its performance with existing methods. Furthermore, the performance of ADINA is evaluated on deep learning tasks through implementation in TensorFlow [17] and comparison with other optimization methods available in the TensorFlow API's Keras Optimizers.

II. BACKGROUND

This section introduces the background algorithms: DIN [8], INNA [11], and Adam [2] and its dynamical system model [12].

A. Dynamical Inertial Newton (DIN)

DIN is an optimization algorithm that combined Newton method with momentum in a continuous-time dynamical systems framework, as proposed in [8]–[10] as a spatiotemporal second-order dynamical model. DIN refers to a dynamical model that combines two types of second-order derivatives: the temporal second-order derivative derived from the momentum method shown as $\ddot{x}(t) = d^2x(t)/dt^2$, and the spatial second-order derivative (Hessian) in the parameter space derived from the Newton method given by $\nabla^2 f(x) = \partial^2 f(x)/\partial x^2$. DIN is formulated as follows:

The k -th iteration of the Newton method can be expressed as (3).

$$\nabla^2 f(x_k)(x_{k+1} - x_k) + \nabla f(x_k) = 0. \quad (3)$$

The dynamical model representation of (3) is given by

$$b\nabla^2 f(x(t))\dot{x}(t) + \nabla f(x(t)) = 0, \quad (4)$$

where $t \in [0, \infty)$, and b is considered as a time step. The relation $(x_{k+1} - x_k)/b \simeq \dot{x}(t)$ holds under this interpretation [10]. The momentum method, also known as the Heavy Ball

method, is a first-order optimizer in a dynamical system [7]. It is described as

$$\ddot{x}(t) + a\dot{x}(t) + \nabla f(x(t)) = 0. \quad (5)$$

By combining (4) and (5), DIN is formulated as

$$\ddot{x}(t) + a\dot{x}(t) + b\nabla^2 f(x(t))\dot{x}(t) + \nabla f(x(t)) = 0, \quad (6)$$

where a and b are hyperparameters [8]. DIN provides powerful optimization capabilities by utilizing the Hessian $\nabla^2 f(x(t))$, which requires significant computational resources.

B. INertia Newton Algorithm

INNA was derived from DIN [8] by transforming its second-order ordinary differential equation into a system of first-order ordinary differential equations through the introduction of an auxiliary variable vector of $y(t) \in \mathbb{R}^d$ [8], [11]. This transformation enables elimination of the Hessian term. The resulting first-order ordinary differential equations without the Hessian term are shown in (7).

$$\begin{cases} \dot{x}(t) + b\nabla f(x(t)) + (a - \frac{1}{b})x(t) + \frac{1}{b}y(t) = 0, \\ \dot{y}(t) + (a - \frac{1}{b})x(t) + \frac{1}{b}y(t) = 0. \end{cases} \quad (7)$$

INNA [11] was obtained by discretizing (7) with a time step η_k using explicit Euler method, as shown in (8).

$$\begin{cases} x_{k+1} = x_k + \eta_k \left[\left(\frac{1}{b} - a \right) x_k - \frac{1}{b} y_k - b\nabla f(x_k) \right], \\ y_{k+1} = y_k + \eta_k \left[\left(\frac{1}{b} - a \right) x_k - \frac{1}{b} y_k \right]. \end{cases} \quad (8)$$

Here, k is the iteration count. The hyperparameters a and b are positive constants whose recommended values were determined by the specific optimization problem being solved. While INNA successfully reduced the computational cost of DIN and made it applicable to NN training, its convergence speed in NN optimization remains slow, and the momentum term was not effectively utilized.

C. Adam and its Dynamical Model

Adam [2] is one of the most widely used optimization algorithms due to its efficient deep-learning capabilities. Adam combines the adaptive learning rate mechanism of RMSprop [1] with the momentum method, incorporating bias correction for both components. The updated equations of Adam were given by

$$m_{k+1} = \beta_1 m_k + (1 - \beta_1) \nabla f(x_k), \quad (9)$$

$$v_{k+1} = \beta_2 v_k + (1 - \beta_2) (\nabla f(x_k))^2, \quad (10)$$

$$x_{k+1} = x_k - \eta_k \frac{m_{k+1} / (1 - \beta_1^{k+1})}{\sqrt{v_{k+1} / (1 - \beta_2^{k+1})} + \epsilon}, \quad (11)$$

where k is the iteration count, and $(\nabla f(x_k))^2$ represents the element-wise square of the gradient vector. η_k denotes the learning rate and ϵ is a parameter to avoid dividing by zero. The hyperparameters $\beta_1, \beta_2 \in [0, 1]$ control the exponential decay rates for the moment estimates. The recommended values are $\eta_k = 10^{-3}$, $\epsilon = 10^{-7}$ or 10^{-8} , $\beta_1 = 0.9$, and

$\beta_2 = 0.999$ [2]. Recent research has focused on analyzing Adam through a dynamical systems perspective. The discrete update equations (9)-(11) can be represented as the following system of ordinary differential equations [12]:

$$\dot{m}(t) = \nu (\nabla f(x(t)) - m(t)), \quad (12)$$

$$\dot{v}(t) = \mu \left((\nabla f(x(t)))^2 - v(t) \right), \quad (13)$$

$$\dot{x}(t) = -\frac{(1 - e^{-\nu t})^{-1} m(t)}{\sqrt{(1 - e^{-\mu t})^{-1} v(t) + \epsilon}}. \quad (14)$$

When these continuous-time equations are discretized using the explicit Euler method with time step η_k , the damping coefficients μ and ν correspond to Adam's parameters β_1 and β_2 through the relationships $\beta_1 = 1 - \mu\eta_k$ and $\beta_2 = 1 - \nu\eta_k$, respectively. Thus, (12)-(14) represent a dynamical momentum model incorporating bias correction and adaptive learning rate updates based on the squared gradient.

III. ADAPTIVE MOMENTUM ESTIMATION IN DIN ALGORITHM

This paper focuses on the possibility of a novel transformation into a system of first-order ODEs by introducing an additional variable from DIN. While similar to INNA, the key distinction lies in the transformation's ability to incorporate bias correction of momentum coefficients and adaptive learning rates used in Adam's dynamical model. This algorithm is called Adaptive momentum estimation in DIN Algorithm (ADINA). By combining the advantages of Newton-based methods and Adam's adaptive mechanisms, ADINA demonstrates a rapid escape from plateau regions and achieves higher accuracy solutions in less time than other first-order methods.

A. Derivation of ADINA

First, we revisit the momentum method of (5).

$$\ddot{x}(t) + a\dot{x}(t) + \nabla f(x(t)) = 0. \quad (5)$$

It can also be expressed in an equivalent form by introducing an auxiliary variable $n(t) \in \mathbb{R}^d$ as shown in (15).

$$\begin{aligned} \dot{n}(t) &= a \left(\frac{1}{a} \nabla f(x(t)) - n(t) \right), \\ \dot{x}(t) &= -n(t). \end{aligned} \quad (15)$$

This means that second-order ODE can be transformed into a system of first-order ODEs by adding one variable. Furthermore, it is assumed that the proposed ADINA has the condition of $1 - ab > 0$ for the hyperparameters a and b in DIN of (6). Similar to the above, a new variable vector $m(t) \in \mathbb{R}^d$ is introduced that satisfies (16).

$$\dot{x}(t) + b\nabla f(x(t)) + m(t) = 0. \quad (16)$$

By differentiating (16) with respect to t , (17) is derived.

$$\ddot{x}(t) + b\nabla^2 f(x(t))\dot{x}(t) + \dot{m}(t) = 0. \quad (17)$$

By subtracting (17) from DIN of (6), (18) is obtained as

$$a\dot{x}(t) + \nabla f(x(t)) - \dot{m}(t) = 0. \quad (18)$$

Furthermore, adding (16) to (18) yields (19).

$$\dot{m}(t) + am(t) + (ab - 1)\nabla f(x(t)) = 0. \quad (19)$$

From (16) and (19), a system of first-order ODEs is derived as,

$$\begin{cases} \dot{m}(t) = a \left(\left(\frac{1-ab}{a} \right) \nabla f(x(t)) - m(t) \right), \\ \dot{x}(t) = -m(t) - b\nabla f(x(t)), \end{cases} \quad (20)$$

where $1 - ab > 0$. In (20), the terms $1 - ab$ and $-b\nabla f(x(t))$ have been added to upper and lower ODEs of (15) respectively. This modification shows how the second-order method influences the first-order method. Furthermore, unlike INNA, this formulation allows for the incorporation of bias correction and adaptive learning rates as seen in Adam's update (12)-(14). As a result, the dynamical model of ADINA is proposed as (21).

$$\begin{cases} \dot{m}(t) = a \left(\left(\frac{1-ab}{a} \right) \nabla f(x(t)) - m(t) \right), \\ \dot{v}(t) = c (\nabla f(x(t))^2 - v(t)), \\ \dot{x}(t) = -\frac{(m(t)/(1 - e^{-at})) + b\nabla f(x(t))}{\sqrt{v(t)/(1 - e^{-ct}) + \epsilon}}. \end{cases} \quad (21)$$

Discretizing (21) with time step η_k using the explicit Euler method yields ADINA as follows:

$$\begin{cases} m_{k+1} = \beta_1 m_k + \hat{\beta}_1 \gamma \nabla f(x_k), \\ v_{k+1} = \beta_2 v_k + \hat{\beta}_2 (\nabla f(x_k))^2, \\ x_{k+1} = x_k - \eta_k \frac{(m_{k+1}/\hat{\beta}_1) + b\nabla f(x_k)}{\sqrt{v_{k+1}/\hat{\beta}_2 + \epsilon}}, \end{cases} \quad (22)$$

where $\gamma = (1 - ab)/a$, $\beta_1 = 1 - \eta_k a$, $\beta_2 = 1 - \eta_k c$, $\hat{\beta}_1 = 1 - \beta_1$, $\hat{\beta}_2 = 1 - \beta_2$, $\hat{\beta}_1 = 1 - \beta_1^{k+1}$, and $\hat{\beta}_2 = 1 - \beta_2^{k+1}$. In (22), β_1 , β_2 , a , b , ϵ and η_k are hyperparameters. Here, the recommended values of Adam [1] are used in ADINA as $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$ or 10^{-8} and $\eta_k = 10^{-3}$. The effectiveness of the proposed method (22) is evaluated for several benchmark optimization problems such as function optimization, logistic regression, classification problems, and deep learning using TensorFlow. The discussion of the specific hyperparameters a and b used in ADINA is presented at the end of the experiments. The complete ADINA procedure is summarized in Algorithm 1. ADINA introduces two additional parameters, a and b , without increasing the number of variable vectors compared to Adam. That is, the computational resources are the same as Adam.

Algorithm 1: ADINA

Required: $\beta_1 = 0.9, \beta_2 = 0.999, \eta = 10^{-3}, \epsilon = 10^{-7}$,
hyperparameters a and b ,
and termination condition λ

Initialize: x_0 by uniform random numbers, $m_0 = 0$,
 $v_0 = 0$ and $k = 0$

1. **While** ($\|\nabla f(x_k)\| > \lambda$ and $k < k_{max}$)
 - (a) Calculate $\nabla f(x_k)$;
 - (b) Update $m_{k+1} = \beta_1 m_k + \hat{\beta}_1 \gamma \nabla f(x_k)$;
 - (c) Update $v_{k+1} = \beta_2 v_k + \hat{\beta}_2 (\nabla f(x_k))^2$;
 - (d) Update $x_{k+1} = x_k - \eta \frac{(m_{k+1}/\hat{\beta}_1) + b \nabla f(x_k)}{\sqrt{v_{k+1}/\hat{\beta}_2 + \epsilon}}$;
 - (e) $k = k + 1$
 2. **Return:** x_k ;
-

B. Convergence Properties for ADINA

The convergence properties of the proposed ADINA are shown as follows:

Definition 1. Let $f(x)$ be Lipschitz continuous. Thus, for any $(x, y) \in \mathbb{R}^d$, (23) is satisfied:

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|_2^2, \quad (23)$$

where $\langle \cdot, \cdot \rangle$ denotes the inner product and L is the Lipschitz constant.

Definition 2. The ℓ_1 and ℓ_2 norms of the gradient $\nabla f(x_k)$ are bounded. That is, there exists a constant G satisfying both of $\|\nabla f(x_k)\|_1 \leq G$ and $\|\nabla f(x_k)\|_2 \leq G$ for $\forall k$. Additionally, the absolute value of each element of $\nabla f(x_k)$ also satisfies (24).

$$g \leq |\nabla f(x_k)_i| \leq G \quad \text{for} \quad \forall k, i. \quad (24)$$

Assume that $v_{k,i}$ also satisfies (25).

$$g^2 \leq v_{k,i} < \frac{v_{k,i}}{\beta_2} < G^2 \quad \text{for} \quad \forall k, i. \quad (25)$$

Definition 3. For the learning rate $\eta_k > 0$ and hyperparameter $b > 0$, (26) is satisfied.

$$\eta_k b L < 2(g + \epsilon). \quad (26)$$

Lemma 1. Let $m_{k+1,i}$ be the i -th element of m_{k+1} . Then, the following inequality holds:

$$|m_{k+1,i}| \leq \gamma G \tilde{\beta}_1, \quad (27)$$

where $\gamma = \frac{1-a}{a}$ and $\tilde{\beta}_1 = 1 - \beta_1^{k+1}$.

Proof of Lemma 1. From (22)

$$\begin{aligned} m_{k+1} &= \beta_1 m_k + \hat{\beta}_1 \gamma \nabla f(x_k) \\ &= \beta_1 \left(\beta_1 m_{k-1} + \hat{\beta}_1 \gamma \nabla f(x_{k-1}) \right) + \hat{\beta}_1 \gamma \nabla f(x_k) \\ &\vdots \\ &= \beta_1^{k+1} m_0 + \hat{\beta}_1 \gamma \sum_{j=0}^k \beta_1^j \nabla f(x_{k-j}). \end{aligned} \quad (28)$$

From the ADINA algorithm shown in Algorithm 1, $m_0 = 0$. Then

$$m_{k+1} = \hat{\beta}_1 \gamma \sum_{j=0}^k \beta_1^j \nabla f(x_{k-j}). \quad (29)$$

Now consider the i^{th} element of m_{k+1} and $\nabla f(x_{k-j})$ as:

$$m_{k+1,i} = \hat{\beta}_1 \gamma \sum_{j=0}^k \beta_1^j \nabla f(x_{k-j})_i. \quad (30)$$

Applying the absolute value to both sides, (31) can be obtained as

$$\begin{aligned} |m_{k+1,i}| &= \hat{\beta}_1 \gamma \left| \sum_{j=0}^k \beta_1^j \nabla f(x_{k-j})_i \right| \\ &\leq \hat{\beta}_1 \gamma \sum_{j=0}^k \beta_1^j |\nabla f(x_{k-j})_i|. \end{aligned} \quad (31)$$

From Definition 2,

$$\begin{aligned} &\leq \hat{\beta}_1 \gamma G \sum_{j=0}^k \beta_1^j \\ &= \hat{\beta}_1 \gamma G \frac{1 - \beta_1^{k+1}}{1 - \beta_1} \\ &= \gamma G \tilde{\beta}_1. \end{aligned} \quad (32)$$

□

Theorem 1. The sequence x_k obtained by ADINA satisfies the following for any integer $k > 0$:

$$\begin{aligned} \|\nabla f(x_k)\|_2^2 &\leq \frac{f(x_k) - f(x_{k+1})}{C_k} \\ &\quad + \gamma \left(\frac{\eta_k G^2}{C_k(g + \epsilon)} \cdot \left| \frac{bL\eta_k}{g + \epsilon} - 1 \right| \right) + \gamma^2 \left(\frac{L\eta_k^2 G^2}{2C_k(g + \epsilon)} \right), \end{aligned} \quad (33)$$

where $C_k = \frac{b\eta_k}{G + \epsilon} \cdot \left(1 - \frac{bL\eta_k}{2(g + \epsilon)} \right)$.

Corollary 1. Here, if we set the learning rate η_k to be constant with respect to iteration k , that is, setting it as $\eta_k = \eta$, then

$$\begin{aligned} \|\nabla f(x_k)\|_2^2 &\leq \frac{f(x_k) - f(x_{k+1})}{C} \\ &\quad + \gamma \left(\frac{\eta G^2}{C(g + \epsilon)} \cdot \left| -1 + \frac{bL\eta}{g + \epsilon} \right| \right) + \gamma^2 \left(\frac{L\eta^2 G^2}{2C(g + \epsilon)^2} \right), \end{aligned} \quad (34)$$

where $C = \frac{b\eta}{G + \epsilon} \left(1 - \frac{bL\eta}{2(g + \epsilon)} \right)$.

Taking the average over k as $[0 \sim K - 1]$,

$$\begin{aligned} \frac{1}{K} \sum_{k=0}^{K-1} \|\nabla f(x_k)\|_2^2 &\leq \frac{f(x_0) - f(x_{K-1})}{KC} \\ &\quad + \gamma \left(\frac{\eta G^2}{C(g + \epsilon)} \cdot \left| -1 + \frac{bL\eta}{g + \epsilon} \right| \right) + \gamma^2 \left(\frac{L\eta^2 G^2}{2C(g + \epsilon)^2} \right). \end{aligned} \quad (35)$$

Here, assuming $K \rightarrow \infty$ to be,

$$\lim_{K \rightarrow \infty} \frac{1}{K} \sum_{k=0}^{K-1} \|\nabla f(x_k)\|_2^2 \leq \gamma \left(\frac{\eta G^2}{C(g+\epsilon)} \cdot \left| -1 + \frac{bL\eta}{g+\epsilon} \right| \right) + \gamma^2 \left(\frac{L\eta^2 G^2}{2C(g+\epsilon)^2} \right). \quad (36)$$

Corollary 2. When parameters a and b satisfy $ab = 1$, then with $\gamma = \frac{1-ab}{a} = 0$, (36) yields (37).

$$\lim_{K \rightarrow \infty} \frac{1}{K} \sum_{k=0}^{K-1} \|\nabla f(x_k)\|_2^2 = 0. \quad (37)$$

Note that when $ab = 1$, the update of momentum term m_k becomes ineffective, nullifying the acceleration benefits of the momentum. Consequently, by choosing parameters a and b that satisfy $ab \approx 1$, ADINA becomes RMSprop with the bias correction. Thus, ADINA compensates for convergence but loses momentum acceleration performance.

Proof of Theorem 1. From Definition 1, for any iteration k , the objective function $f(x)$ holds as (38):

$$f(x_{k+1}) \leq f(x_k) + \langle \nabla f(x_k), x_{k+1} - x_k \rangle + \frac{L}{2} \|x_{k+1} - x_k\|_2^2. \quad (38)$$

Therefore, by the iteration formula of ADINA, (38) becomes

$$f(x_{k+1}) \leq f(x_k) - \eta_k \langle \nabla f(x_k), \frac{\hat{m}_{k+1} + b\nabla f(x_k)}{\sqrt{\hat{v}_{k+1}} + \epsilon} \rangle + \frac{L\eta_k^2}{2} \cdot \left\| \frac{\hat{m}_{k+1} + b\nabla f(x_k)}{\sqrt{\hat{v}_{k+1}} + \epsilon} \right\|_2^2, \quad (39)$$

where $\hat{m}_{k+1} = m_{k+1}/\tilde{\beta}_1$ and $\hat{v}_{k+1} = v_{k+1}/\tilde{\beta}_2$. Let the i -th elements of $\hat{m}_{k+1}$, $\hat{v}_{k+1}$, and $\nabla f(x_k)$ to be $\hat{m}_{k+1,i}$, $\hat{v}_{k+1,i}$, and $\nabla f(x_k)_i$, respectively, (39) can be transformed as

$$f(x_{k+1}) \leq f(x_k) - \eta_k \sum_{i=1}^d \frac{\hat{m}_{k+1,i} \nabla f(x_k)_i + b(\nabla f(x_k)_i)^2}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} + \frac{L\eta_k^2}{2} \sum_{i=1}^d \frac{(\hat{m}_{k+1,i})^2 + 2b\hat{m}_{k+1,i} \nabla f(x_k)_i + b^2(\nabla f(x_k)_i)^2}{(\sqrt{\hat{v}_{k+1,i}} + \epsilon)^2} \quad (40)$$

$$= f(x_k) - \eta_k \sum_{i=1}^d \frac{b(\nabla f(x_k)_i)^2}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} + \frac{L\eta_k^2}{2} \sum_{i=1}^d \frac{b^2(\nabla f(x_k)_i)^2}{(\sqrt{\hat{v}_{k+1,i}} + \epsilon)^2} \quad (40.1)$$

$$- \eta_k \sum_{i=1}^d \frac{\hat{m}_{k+1,i} \nabla f(x_k)_i}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} + L\eta_k^2 \sum_{i=1}^d \frac{b\hat{m}_{k+1,i} \nabla f(x_k)_i}{(\sqrt{\hat{v}_{k+1,i}} + \epsilon)^2} \quad (40.2)$$

$$+ \frac{L\eta_k^2}{2} \sum_{i=1}^d \frac{(\hat{m}_{k+1,i})^2}{(\sqrt{\hat{v}_{k+1,i}} + \epsilon)^2}. \quad (40.3)$$

From $g^2 \leq \hat{v}_{k+1,i}$ in Definition 2, (40.1) is transformed as

$$(40.1) \leq f(x_k) - b\eta_k \sum_{i=1}^d \frac{(\nabla f(x_k)_i)^2}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} + \frac{b^2 L\eta_k^2}{2} \sum_{i=1}^d \frac{(\nabla f(x_k)_i)^2}{(g+\epsilon)(\sqrt{\hat{v}_{k+1,i}} + \epsilon)} \quad (40.1.1)$$

$$= f(x_k) - \left(b\eta_k - \frac{b^2 L\eta_k^2}{2(g+\epsilon)} \right) \sum_{i=1}^d \frac{(\nabla f(x_k)_i)^2}{\sqrt{\hat{v}_{k+1,i}} + \epsilon}. \quad (40.1.2)$$

Here, from Definition 3, and $\hat{v}_{k+1,i} \leq G^2$ in Definition 2,

$$(40.1.2) \leq f(x_k) - \left(b\eta_k - \frac{b^2 L\eta_k^2}{2(g+\epsilon)} \right) \sum_{i=1}^d \frac{(\nabla f(x_k)_i)^2}{G+\epsilon} \quad (40.1.3)$$

$$= f(x_k) - \frac{b\eta_k}{G+\epsilon} \left(1 - \frac{bL\eta_k}{2(g+\epsilon)} \right) \|\nabla f(x_k)\|_2^2, \quad (40.1.4)$$

where from Definition 3, $1 - \frac{bL\eta_k}{2(g+\epsilon)} > 0$. Next, we consider about (40.2).

$$(40.2) = -\eta_k \sum_{i=1}^d \frac{\hat{m}_{k+1,i} \nabla f(x_k)_i}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} + L\eta_k^2 \sum_{i=1}^d \frac{b\hat{m}_{k+1,i} \nabla f(x_k)_i}{(\sqrt{\hat{v}_{k+1,i}} + \epsilon)^2} \quad (40.2.1)$$

$$= \eta_k \sum_{i=1}^d \left(\frac{\hat{m}_{k+1,i} \nabla f(x_k)_i}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} \left(-1 + \frac{bL\eta_k}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} \right) \right) \quad (40.2.2)$$

$$\leq \eta_k \sum_{i=1}^d \left(\left| \frac{\hat{m}_{k+1,i} \nabla f(x_k)_i}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} \right| \cdot \left| -1 + \frac{bL\eta_k}{\sqrt{\hat{v}_{k+1,i}} + \epsilon} \right| \right). \quad (40.2.3)$$

From $g^2 \leq \hat{v}_{k+1,i}$ in Definition 2 and $|m_{k+1,i}| \leq \gamma G \tilde{\beta}_1$ in Lemma 1,

$$|\hat{m}_{k+1,i}| = \frac{|m_{k+1,i}|}{\tilde{\beta}_1} \leq \frac{\gamma G \tilde{\beta}_1}{\tilde{\beta}_1} = \gamma G. \quad (40.2.4)$$

Therefore,

$$(40.2.3) \leq \eta_k \sum_{i=1}^d \left(\frac{\gamma G |\nabla f(x_k)_i|}{g+\epsilon} \cdot \left| -1 + \frac{bL\eta_k}{g+\epsilon} \right| \right) \quad (40.2.5)$$

$$= \frac{\eta_k \gamma G}{g+\epsilon} \left| -1 + \frac{bL\eta_k}{g+\epsilon} \right| \sum_{i=1}^d |\nabla f(x_k)_i|. \quad (40.2.6)$$

Furthermore, from $\|\nabla f(x_k)\|_1 \leq G$ in Definition 2,

$$(40.2.6) \leq \frac{\eta_k \gamma G^2}{g+\epsilon} \left| -1 + \frac{bL\eta_k}{g+\epsilon} \right|. \quad (40.2.7)$$

In the last, we consider about (40.3). From $g^2 \leq \hat{v}_{k+1,i}$ in Definition 2 and (40.2.4),

$$(40.3) = \frac{L\eta_k^2}{2} \sum_{i=1}^d \frac{(\hat{m}_{k+1,i})^2}{(\sqrt{\hat{v}_{k+1,i}} + \epsilon)^2} \leq \gamma^2 \left(\frac{L\eta_k^2 G^2}{2(g+\epsilon)^2} \right). \quad (40.3.1)$$

From (40.1.4), (40.2.7) and (40.3.1), (40) becomes (41).

$$f(x_{k+1}) \leq f(x_k) - \frac{b\eta_k}{G+\epsilon} \left(1 - \frac{bL\eta_k}{2(g+\epsilon)}\right) \|\nabla f(x_k)\|_2^2 + \gamma \left(\frac{\eta_k G^2}{g+\epsilon} \left| -1 + \frac{bL\eta_k}{g+\epsilon} \right| \right) + \gamma^2 \left(\frac{L\eta_k^2 G^2}{2(g+\epsilon)^2} \right). \quad (41)$$

Here, let $C_k = \frac{b\eta_k}{G+\epsilon} \left(1 - \frac{bL\eta_k}{g+\epsilon}\right)$ be,

$$\|\nabla f(x_k)\|_2^2 \leq \frac{f(x_k) - f(x_{k+1})}{C_k} + \gamma \left(\frac{\eta_k G^2}{g+\epsilon} \left| -1 + \frac{bL\eta_k}{g+\epsilon} \right| \right) + \gamma^2 \left(\frac{L\eta_k^2 G^2}{2(g+\epsilon)^2} \right). \quad (42)$$

This concludes the proof. $\square$

IV. EXPERIMENTS

This section evaluates ADINA's performance through four benchmark problems: function optimization, regression, and classification problems, and deep learning tasks. For the first three problems, experiments were conducted using Numpy on Google Colab, comparing ADINA with the Newton method, Adam, and INNA. Note that the standard Newton method may diverge due to excessively large update steps. Therefore, in experiments, the Damped Newton method [14] was used as the Newton method, and the step size of the Newton method was set to $\eta_k = 10^{-1}$ for all experiments. Furthermore, the proposed ADINA was implemented as a custom Keras optimizer in TensorFlow API [17] and evaluated against standard optimizers, including SGD, AdaGrad, RMSprop, and Adam, in deep learning experiments. The hyperparameters for each algorithm were set as follows (AdaGrad: $\epsilon = 10^{-7}$; RMSprop: $\beta = 0.9$, $\epsilon = 10^{-7}$; Adam: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$; INNA: $a = 0.1$, $b = 0.1$; ADINA: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $a = 0.1$, $b = 0.9$, $\epsilon = 10^{-7}$). AdaGrad, RMSprop, Adam, and INNA hyperparameters were set according to their papers and Keras documentation. The experiments demonstrate ADINA's effectiveness in traditional optimization tasks and modern deep learning scenarios, for both batch and mini-batch training. All results presented here represent the best achieved performance for each benchmark problem. The impact of ADINA's key hyperparameters a and b on optimization performance is discussed at the end of this section.

A. Function Optimization Problem: Rosenbrock Function

The first computer experiment is a function optimization problem using the Rosenbrock function (also known as the banana function) [15] in (43).

$$f(x) = \sum_{i=1}^{d-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2]. \quad (43)$$

The initial point for optimization is set as $x_0 = (x_{0,i})_{i=1}^d$ where $x_{0,i} = -1.2$ if i is odd and $x_{0,i} = 1.0$ if i is even. The Rosenbrock function has a global minimum solution at

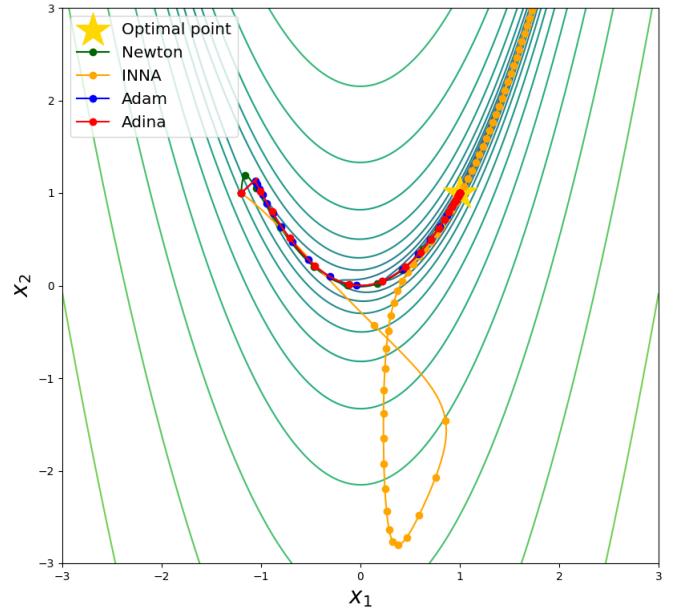


Fig. 1: Optimization trajectories of $d = 2$ Rosenbrock Function.

$x^* = (1, 1, \dots, 1)^T \in \mathbb{R}^d$ with $f(x^*) = 0$, and notably contains no local minima except for this global minimum. The experiments are conducted with varying dimensions $d = 2, 100, 1000, 10,000$. Each optimization process is limited to a maximum of $k_{max} = 10,000$ iterations with a termination condition of $\|\nabla f(x_k)\| < 10^{-6}$. For INNA, Adam, and ADINA, utilize a consistent step size of $\eta = 10^{-3}$. The experimental results are summarized in Table I. The Newton method is the fastest lower-dimensional convergence by exploiting exact Hessian effects. However, it fails to converge for $d = 10,000$ due to computer resource limitations in the experimental environments. ADINA converges faster, while Adam requires maximum iterations to approach the optimal solution. The enhanced performance of ADINA stems from four fundamental mechanisms: three Adam's intrinsic prop-

TABLE I: Rosenbrock Function Results

Dimension (d)	Algorithm	Iterations (k)	Final Value $f(x)$
2	Newton	222	1.612×10^{-13}
	INNA	6,825	4.197×10^{-2}
	Adam	10,000	3.678×10^{-4}
	ADINA	4,746	7.207×10^{-9}
100	Newton	241	1.471×10^{-13}
	INNA	10,000	2.098
	Adam	10,000	1.839×10^{-2}
	ADINA	5,118	4.877×10^{-9}
1,000	Newton	252	1.448×10^{-13}
	INNA	10,000	2.098×10^1
	Adam	10,000	1.839×10^{-1}
	ADINA	5,305	3.972×10^{-9}
10,000	Newton	N/A	N/A
	INNA	10,000	2.098×10^2
	Adam	10,000	1.839
	ADINA	5,474	3.280×10^{-9}

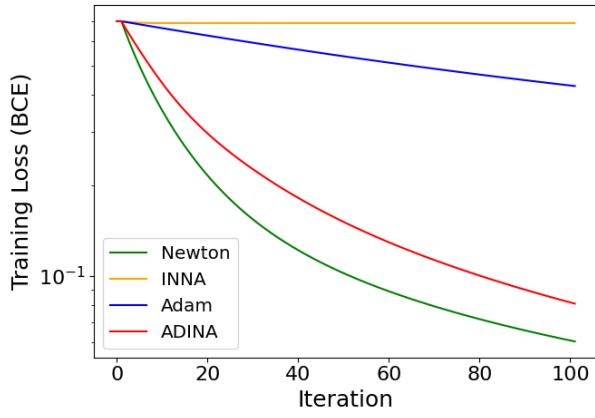


Fig. 2: Evolution of training loss over iterations for the mushroom dataset.

erties, which are bias correction, effective momentum term, and adaptive learning rate adjustment, and ADINA's unique feature of the effect of Hessian condensed into an added variable utilization. A comparative analysis between INNA and ADINA reveals that although both methods leverage Hessian effects, ADINA demonstrates accelerated convergence (approximately half the iterations) and achieves final values closer to the optimal solution (order of 10^{-9}). Figure 1 shows the optimization trajectories for $d = 2$ of the Rosenbrock function. The INNA oscillates near the curved valley, while the Newton method (here, the Damped Newton method), Adam, and ADINA follow more stable paths to the optimal solution. Although Adam maintains stability, it requires significantly more iterations than ADINA, Newton method, and INNA, demonstrating the effectiveness of utilizing Hessian effects in optimization processes.

B. Logistic Regression

For the experimental evaluation of logistic regression, the mushroom dataset [16] from the Audubon Society Field Guide

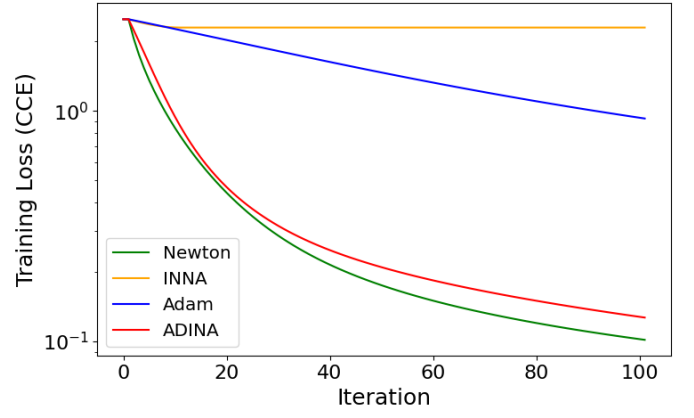


Fig. 4: Evolution of training loss over iterations for the 8×8 MNIST dataset.

served as test data. This dataset contains 8,124 samples of gilled mushrooms from the Agaricus and Lepiota Family, each characterized by 22 categorical features describing their physical attributes. Each mushroom is classified as edible (1) or poisonous (0). The dataset was randomly split into a training set of 6,499 samples (80%) and a test set of 1,625 samples (20%). The objective function of logistic regression is binary cross-entropy of (44).

$$f(x) = -\frac{1}{P} \sum_{p=1}^P [O_p \log(\text{sig}(x^T I_p)) + (1 - O_p) \log(1 - \text{sig}(x^T I_p))] , \quad (44)$$

where $\text{sig}(\cdot)$ is the sigmoid activation function, P is the number of samples, O_p is the true label (0 or 1), x is the weight parameter vector, and I_p is the input feature vector for sample p . The sigmoid function maps the input to a probability value between 0 and 1. The learning rate was set to $\eta_k = 10^{-3}$ for ADINA, Adam and INNA, with maximum iterations $k_{max} = 100$ and a termination criterion of $\|\nabla f(x_k)\| < 10^{-4}$. The logistic regression results on the mushroom dataset are

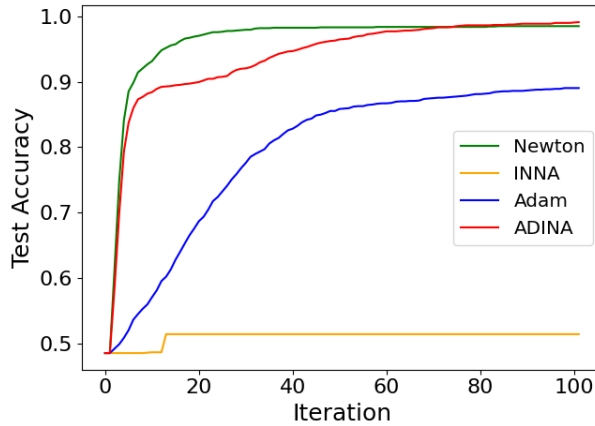


Fig. 3: Evolution of test accuracy over iterations for the mushroom dataset.

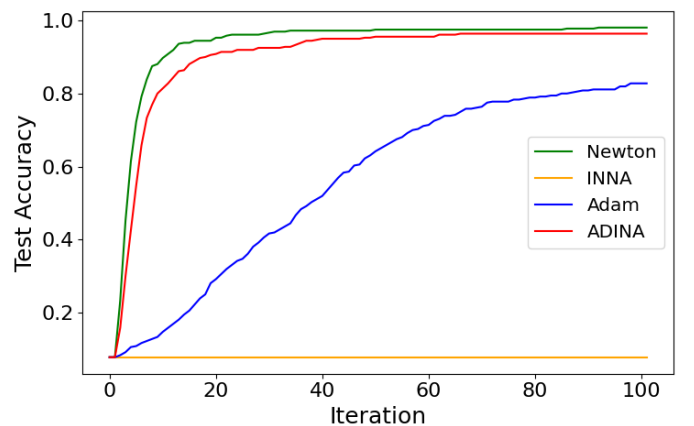


Fig. 5: Evolution of test accuracy over iterations for the 8×8 MNIST dataset.

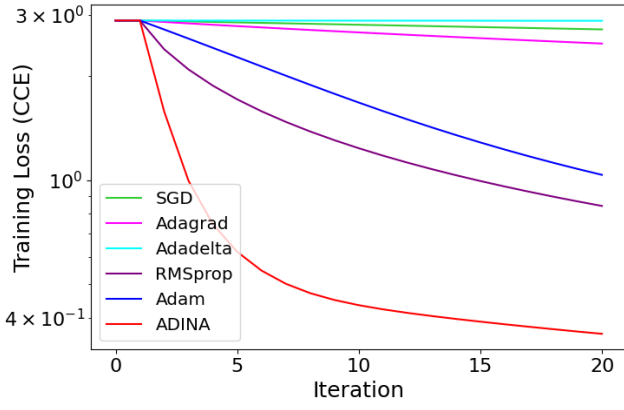


Fig. 6: Evolution of training loss over iterations for batch training of the 28×28 MNIST dataset.

shown in figures 2 and 3. Figure 2 displays the training loss across iterations, while figure 3 shows the test accuracy across iterations. The Newton method, leveraging Hessian effects, achieves superior accuracy and lower loss in early epochs. On the other hand, in terms of test accuracy, ADINA achieves a higher final test accuracy (99.13%) compared to the Newton method (98.52%) in the final stages of training. Among the first-order methods, including INNA, Adam, and the proposed ADINA, ADINA shows rapid convergence to high accuracy and low loss due to its second-order convergence effect. However, INNA's Hessian effects exhibit sensitivity to network adjustments and hyperparameter settings. This sensitivity makes it challenging for INNA to match ADINA's performance in logistic regression when using the hyperparameters specified in [11].

C. Classification Problem: MNIST Handwritten Digits Dataset

The effectiveness of the proposed method was investigated through training on classification tasks using 8×8 and 28×28 MNIST handwritten digits datasets [1]. The network architecture comprises a simple configuration with input and output layers, without hidden layers. The objective function for this problem is the categorical cross-entropy loss with softmax activation, defined as

$$f(x) = -\frac{1}{P} \sum_{p=1}^P \sum_{c=1}^{cls} [O_{pc} \log(\text{softmax}(x^T I_p)_c)]. \quad (45)$$

where P is the total number of samples, cls is the number of classes (10 for digits 0-9), O_{pc} is the true label for sample p and class c (1 if the sample belongs to class c , zero otherwise), x is the weight parameter vector, I_p is the input feature vector for sample p , and $\text{softmax}(\cdot)_c$ is the softmax activation function that outputs the probability for class c . The maximum number of iterations and learning rate were set to $k_{max} = 100$, and $\eta_k = 10^{-3}$, respectively.

1) 8×8 MNIST Handwritten Digits Dataset: The effectiveness of the proposed method was investigated using the

MNIST handwritten digit dataset downsampled to 8×8 pixels [1]. The data dimension is $d = 64$, with 1,797 samples split into 75% training and 25% test sets. Figures 4 and 5 illustrate the optimization characteristics during the training of the 8×8 MNIST classification model. ADINA behaves similarly to Newton's method. Among the first-order methods, ADINA achieves high-precision solutions in the initial stage, demonstrating superior performance. This efficiency can be attributed to its unique approach of incorporating momentum-based adaptation while maintaining stable convergence properties. INNA shows higher sensitivity to hyperparameter settings, resulting in slower convergence compared to both ADINA and Adam. The results highlight ADINA's ability to effectively balance adaptive learning rates with momentum, making it particularly well-suited for the classification task.

2) 28×28 MNIST Handwritten Digits Dataset: Further experiments were conducted using the MNIST handwritten digit dataset with 28×28 pixel resolution [1]. The data dimension is $d = 784$, with standard training and test sets containing 60,000 and 10,000 samples, respectively. Due to the high computational cost of calculating Hessians in high-dimensional parameter spaces, only first-order optimization methods were evaluated. Therefore, the Newton method was excluded from the comparison due to computational resource constraints. The proposed method was implemented as a Keras optimizer in TensorFlow and compared with standard Keras optimizers SGD, AdaGrad, AdaDelta, RMSprop, and Adam in [18]. The default hyperparameter settings provided by Keras were used for the standard optimizers, while ADINA parameters were set to $\beta_1 = 0.9$, $\beta_2 = 0.999$, $a = 0.1$, $b = 0.9$, $\epsilon = 10^{-7}$, and $\eta_k = 10^{-3}$. The ADINA implementation is available at [17]. As shown in Figures 6 and 7, ADINA performs better than other optimizers, exhibiting lower training errors and higher test accuracy in the early stages of training. First-order methods such as SGD, AdaGrad, and AdaDelta require more iterations to achieve solutions comparable to those of other methods. In this batch training context, RM-

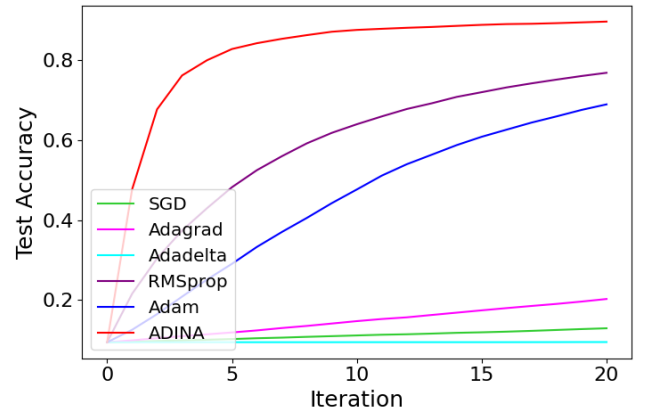


Fig. 7: Evolution of test accuracy over iterations for batch training of the 28×28 MNIST dataset.

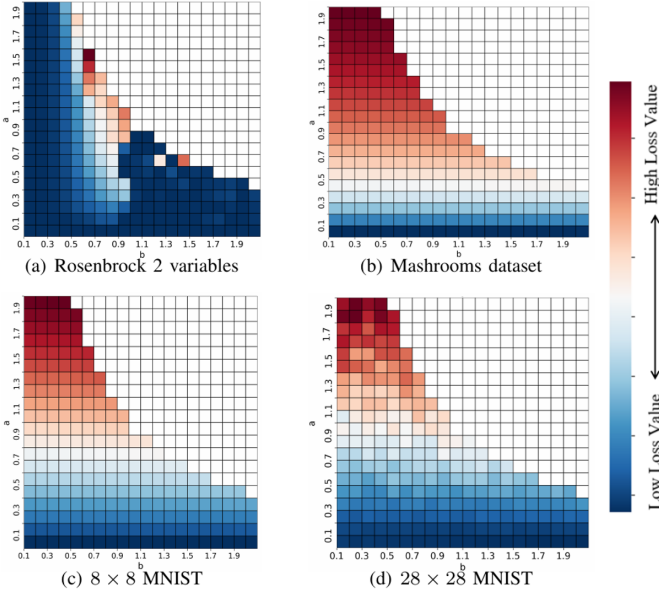


Fig. 8: Variation in training loss due to ADINA’s hyperparameters a and b .

Sprop shows competitive performance in training convergence, indicating that the advantages typically associated with Adam-based optimizers may not be as pronounced in this specific batch training scenario.

Figure 8 shows heatmaps of learning errors across different values of ADINA’s hyperparameters $a, b \in [0.1, 2.0]$ within a limited number of iterations. White regions indicate invalid parameter combinations where $1 - ab < 0$. Darker blue indicates lower error. For the low-dimensional Rosenbrock function ($d = 2$), many combinations of a and b achieve similar low errors, shown by the widespread dark blue regions. However, the optimal parameter region becomes more concentrated as problem dimensionality increases through the Mushrooms dataset, 8×8 MNIST, and 28×28 MNIST. Specifically, $a = 0.1$ consistently yields good performance across different b values in higher dimensions, indicating high robustness. This dimensional dependency of parameter sensitivity suggests that for function optimization, logistic regression, and pattern classification problems in this paper, setting $a = 0.1$ with b in the mid-range provides reliable performance.

D. Deep Learning with mini-batch training

This section evaluates the proposed method’s effectiveness in deep learning using TensorFlow for mini-batch training. The experiments were conducted on the 28×28 MNIST dataset. Based on previous experimental results showing slower convergence of SGD, AdaGrad, and AdaDelta compared to RMSprop and Adam, the comparison focuses on RMSprop, Adam, and ADINA for mini-batch training. Different batch sizes are evaluated using two hidden layers fully connected network with maximum epoch $e_{max} = 20$ and batch sizes (16, 32, 64, 512, 1024, and 2048). Additionally, the effectiveness of the proposed method is examined using a simple CNN architecture

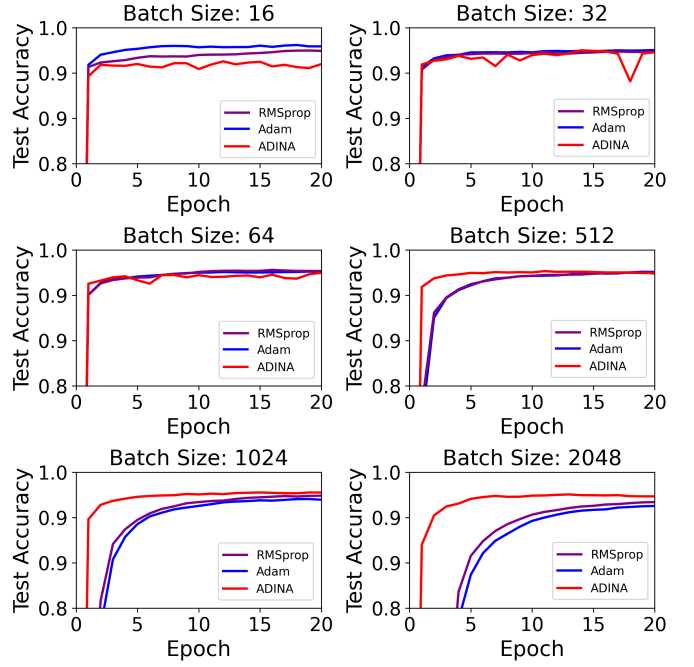


Fig. 9: Comparison of optimizers across different batch sizes for 28×28 MNIST with ADINA’s hyperparameter $a = 0.1$ and $b = 0.9$.

with two convolutional layers followed by one fully connected layer. Note that in mini-batch training, an epoch consists of multiple iterations, where the number of iterations is calculated as $(P/\text{mini-batch}) \times \text{epoch}$, with P representing the total number of training samples. Therefore, smaller batch sizes result in more iterations per epoch. All algorithms use Keras default hyperparameters except ADINA, which uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $a = 0.1$, $b = 0.9$, $\epsilon = 10^{-7}$, $\eta_k = 10^{-3}$.

1) *Two-Layer Fully Connected Network*: A comparison of batch sizes was performed using a network consisting of two fully connected layers; each layer has 10 neurons. The network configuration is 784-10-10-10. First, the experimental results for test accuracy are shown in figure 9. The graph shows reduced quadratic convergence effects at small batch sizes for ADINA with parameters $a = 0.1$ and $b = 0.9$, resulting in performance comparable to other examples. Performance improves at larger batch sizes as quadratic convergence strengthens.

2) *Simple CNN*: The experiments used a convolutional neural network architecture consisting of two convolutional layers and a fully connected layer with 128 neurons, implementing a dropout rate of 0.5 and a batch size of 1,024. With ADINA parameters set to $a = 0.1$ and $b = 0.9$, the training graph of figure 10 shows ADINA achieving faster convergence, with the training loss dropping more rapidly in the first five epochs than Adam and RMSprop. The test accuracy graph of figure 11 shows ADINA reaching approximately 98% accuracy earlier

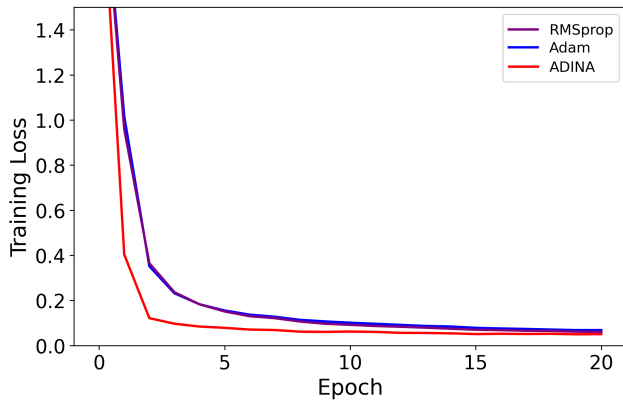


Fig. 10: Evolution of training loss over iterations for CNN mini-batch training of 28×28 .

than the other methods, maintaining a slight edge throughout training. After epoch 15, all three optimizers stabilize at similar performance levels, with test accuracies around 99%. The graphs confirm ADINA's advantage in early-stage optimization while achieving a final performance comparable to that of established optimizers. This means that ADINA is an algorithm that can reduce the objective function with fewer iterations.

V. CONCLUSION

This paper presents ADINA, a novel optimization method combining the advantages of second-order methods and adaptive gradient algorithms. ADINA has reformulated DIN's spatiotemporal second-order dynamical model into a system of first-order ODEs while incorporating the adaptive mechanisms in Adam. The proposed method maintains the beneficial characteristics of second-order methods while achieving computational efficiency by eliminating Hessian calculations. Through comprehensive experiments across function optimization, regression, and classification tasks, we demonstrated that ADINA effectively escapes plateau regions or saddle points

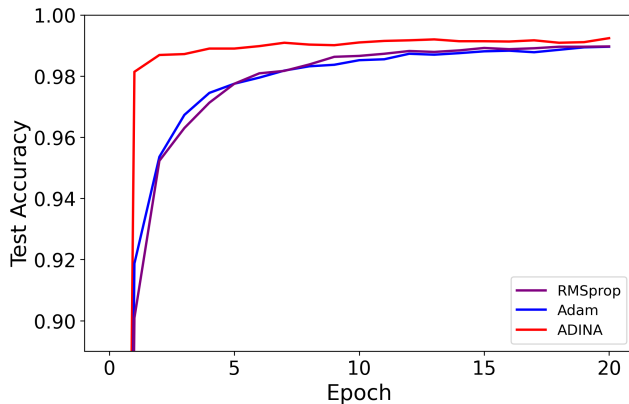


Fig. 11: Evolution of test accuracy over iterations for CNN mini-batch training of the 28×28 MNIST.

and achieves more accurate solutions more rapidly than existing methods. Experimental results based on the TensorFlow implementation demonstrated ADINA's applicability to mini-batch learning. ADINA is derived from a similar dynamical system interpretation as Adam, it is compatible with various improvements applied to Adam. For example, ADINA can be extended to incorporate weight decay, leading to an extended version called ADINA-W. We have implemented ADINA and ADINA-W as Keras optimizers in TensorFlow and released them on GitHub [17].

Future work will focus on (i) theoretical analysis of stochastic properties, (ii) further investigation of convergence behavior, (iii) optimization of hyperparameter settings for mini-batch training, and (iv) large-scale applications of ADINA to deep learning.

ACKNOWLEDGMENT

This work is supported by The Japan Society Promotion of Science (JSPS), KAKENHI (23K11267).

REFERENCES

- [1] I. Goodfellow, et al., *Deep learning*, MIT press, 2016.
- [2] D. P. Kingma, et al., "Adam: A method for stochastic optimization", *Proc. ICLR*, 2015.
- [3] H. Ninomiya, et al., "Robust training of microwave neural network models using combined global/local optimization techniques", *Proc. IEEE MTT-S Int. Microw. Symp. Dig.*, pp. 995–998, 2008.
- [4] J. Nocedal and S. J. Wright, *Numerical optimization*, Springer, 1999.
- [5] A. S. Antipin, "Linearization method, in nonlinear dynamic systems: Qualitative analysis and control", *Proc. ISA Russ. Acad. Sci. (English Translation: Comput. Math.)*, Vol. 2 (8), pp. 4–20 (1–15), 1994 (1997).
- [6] W. Su, et al., "A differential equation for modeling Nesterov's accelerated gradient method: Theory and insights", *Journal on Mach. Learn. Res.*, Vol. 17, pp. 1–43, 2016.
- [7] H. Attouch, et al., "The heavy ball with friction method, I. The continuous dynamical system: global exploration of the local minima of a real-valued function by asymptotic analysis of a dissipative dynamical system", *Commun. Contemp. Math.*, Vol. 2, pp. 1–34, 2000.
- [8] F. Alvarez, et al., "A second-order gradient-like dissipative dynamical system with hessian-driven damping: Application to optimization and mechanics", *Journal de mathématiques pures et appliquées*, Vol. 81, pp. 747–779, 2002.
- [9] H. Attouch, et al., "First-order optimization algorithms via inertial systems with Hessian driven damping", *Math. Program.*, pp. 1–43, 2022.
- [10] S. Adly and H. Attouch, "Accelerated optimization through time-scale analysis of inertial dynamics with asymptotic vanishing and Hessian-driven dampings", *Optimization*, pp. 1–38, 2024.
- [11] C. Castera, et al., "An inertial newton algorithm for deep learning", *Journal on Mach. Learn. Res.*, Vol. 22, 2021.
- [12] C. Ma, et al., "A qualitative study of the dynamic behavior for adaptive gradient algorithms", *Proc. Math. Sci. Mach. Learn. PMLR*, pp. 671–692, 2022.
- [13] M. Zaheer, et al., "Adaptive methods for nonconvex optimization", *Proc. NeurIPS*, Vol. 31, 2018.
- [14] M. A. Gomes-Ruggiero, et al., "Comparing algorithms for solving sparse nonlinear systems of equations", *SIAM Journal on Sci. Stat. Comput.*, Vol. 13, No. 2, pp. 459–483, 1992.
- [15] H. H. Rosenbrock, "An automatic method for finding the greatest or least value of a function", *Computer Journal*, Vol. 3, No.3, pp. 175–184, 1960.
- [16] D. Dua and C. Graff, "UCI Machine Learning Repository," 2019. [Online]. Available: <http://archive.ics.uci.edu/ml> (Last access 27 Jun. 2025).
- [17] <https://github.com/ninomiyalab/ADINA.git> (Last access 29 Apr. 2025).
- [18] Keras Google group, "Keras", [Online]. Available: <https://keras.io/api/optimizers/> (Last access 27 Jun. 2025).